Creating and Manipulating Arrays Using NumPy

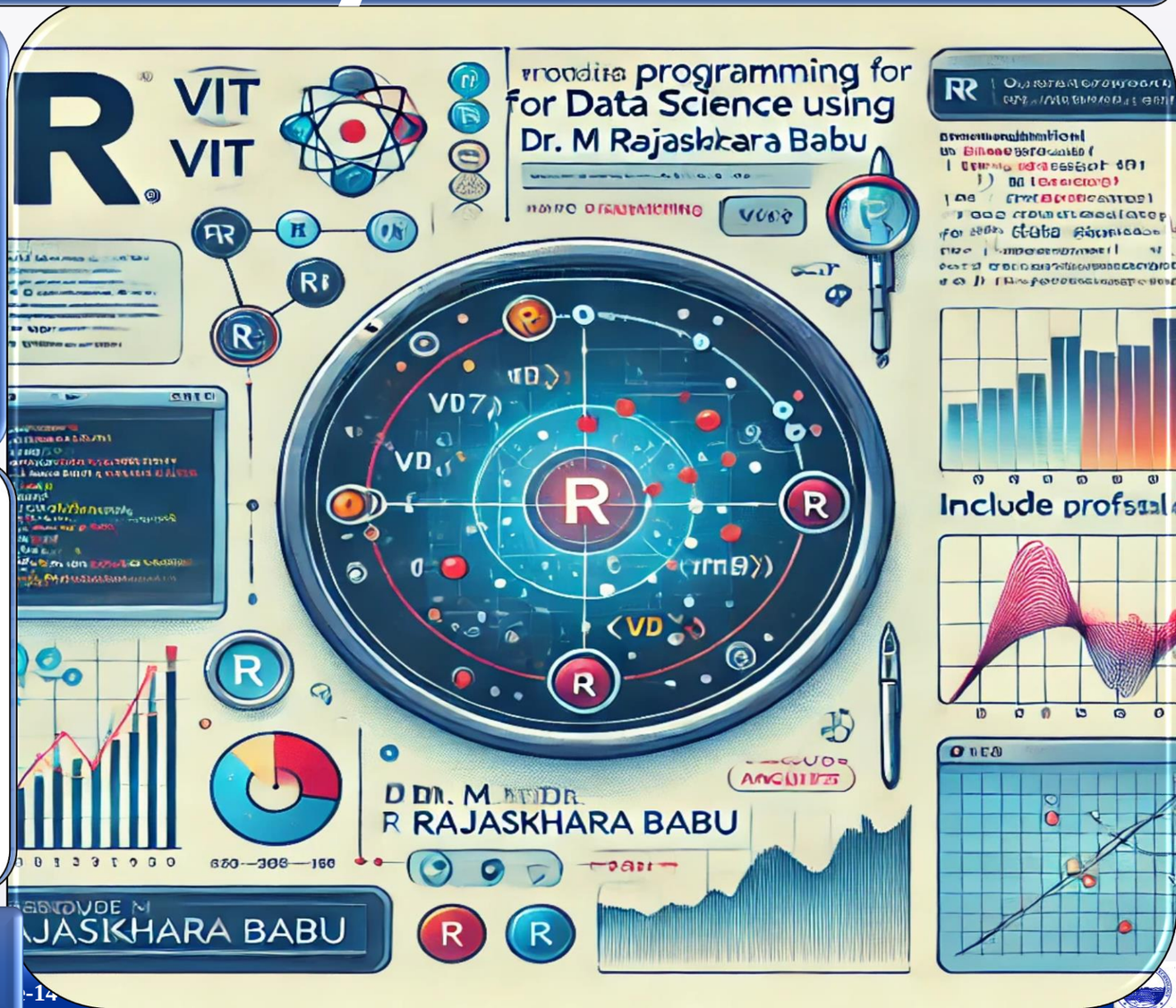
by

Dr M Rajasekhara Babu

School of

Computer Science and Engineering

https://www.rajababu.org



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India

Objectives & Teaching Learning Material

To explain the fundamentals of NumPy arrays and their advantages for efficient numerical computation

To demonstrate the creation, manipulation, and statistical analysis of multidimensional arrays using NumPy functions

To illustrate array operations such as slicing, reshaping, and element-wise computations for data processing applications

Teaching + Learning Material: LCD, White board Marker, Presentation slides

Learning outcomes

Upon successful completion of this lecture, students will be able to:

Create and manage one-dimensional, two-dimensional, and multidimensional NumPy arrays for numerical computing tasks

Apply NumPy functions to perform statistical analysis, indexing, slicing, and reshaping of arrays efficiently

Implement element-wise array operations and utilize NumPy for data science, machine learning, and scientific computing applications

Session Plan

Time in minutes	Topic	Learning Aid & Methodology	Faculty Approach	Student Activity	Skill and Competency Developed
05	Re-Cap	Quiz Presentation	Questions Organizes	Answers Identifies	Knowledge Understanding
10	Introduction to NumPy and Array Fundamentals	Presentation	Presentation	Listens	Remembering
10	Creating Arrays and Performing Statistical Operations	Explains	Explains	Notes	Understanding
15	Array Manipulation through Indexing, Slicing, and Reshaping	Case Study	Organizes	Observes	Applying
10	Element-wise Operations and Real-World Applications	Discussion	Facilitates	Answers	Analyzing

What is NumPy?

NumPy (Numerical Python)

A powerful Python library designed for fast, efficient numerical and scientific computing with multi-dimensional arrays.

Core Use Cases

- Numerical & Scientific Computing
- Matrix Operations
- Data Analysis
- Machine Learning

Key Features

- Fast, vectorized operations
- Multi-dimensional arrays
- Rich mathematical functions

Why NumPy Over Python Lists?

Python List

```
numbers = [1, 2, 3, 4]
```

Standard Python list — flexible but slow for numerical tasks.

NumPy Array

```
import numpy as np
arr = np.array([1, 2, 3, 4])
```

Optimized for speed, memory efficiency, and mathematical operations.

Faster Execution
Vectorized C-level operations

Less Memory
Compact, contiguous storage

Better Math
Built-in statistical functions

Importing NumPy

Standard Import Syntax

```
import numpy as np
```

Why Use the Alias `np`?

Using `np` as an alias makes your code shorter and easier to read. Instead of writing `numpy.array()`, you simply write `np.array()`.

Common NumPy Functions

```
np.array()
np.mean()
np.reshape()
np.std()
np.random.randint()
```

All accessed via the `np` prefix — concise and consistent throughout your code.

What is an Array?

An array is a collection of elements stored in contiguous memory locations, enabling fast and efficient access.

Example

```
arr = np.array([10, 20, 30, 40])
print(arr)
```

Output

```
[10 20 30 40]
```

Key Characteristics

Homogeneous Data

All elements share the same data type

Faster Processing

Contiguous memory enables vectorized ops

Indexed Access

Elements accessed via row/column indices

Dimensions of Arrays

1D Array

```
[10 20 30 40]
```

A single row of elements — like a list.

2D Array

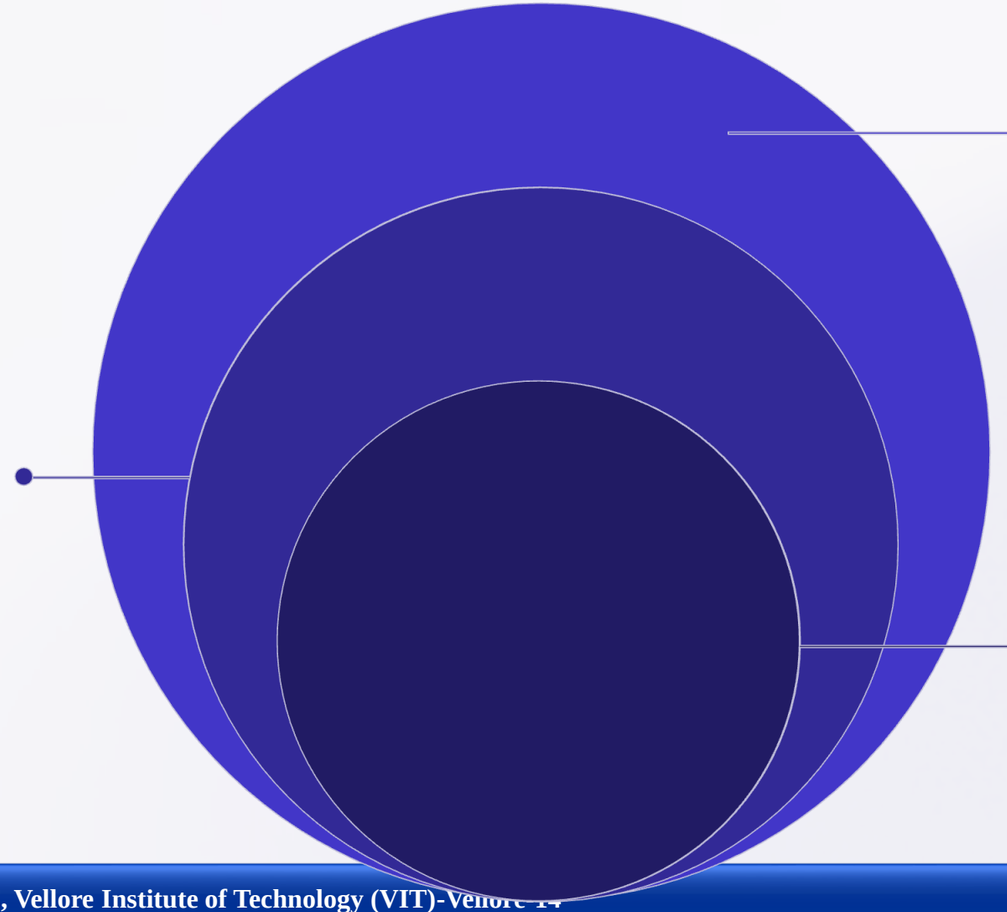
```
[[10 20]
 [30 40]]
```

Rows and columns — like a table or matrix.

3D Array

```
[[[1.2].[3.4]].
 [[5,6],[7,8]]]
```

Multiple 2D matrices stacked together.



2D Array
Rows and columns forming a matrix

3D Array
Stacked 2D matrices forming layers

1D Array
Single row of elements, like a list

Creating Arrays with NumPy

1D Array using `np.array()`

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr)
```

Output

```
[1 2 3 4]
```

2D Array Example

```
arr = np.array([
  [1, 2, 3],
  [4, 5, 6]
])
print(arr)
```

Output

```
[[1 2 3]
 [4 5 6]]
```

- Use `np.array()` to convert any Python list or nested list into a NumPy array. Nested lists become 2D arrays automatically.

Generating Random Arrays

Function: `np.random.randint()`

```
arr = np.random.randint(
  0,
  11,
  size=(3, 4)
)
```

Generates random integers between 0 and 10 in a 3×4 matrix.

Sample Output

```
[[ 2  5  8 10]
 [ 7  1  3  9]
 [ 4  6  2  8]]
```

Understanding Shape

```
arr.shape
# Output: (3, 4)
```

3 rows and 4 columns — shape is always expressed as `(rows, columns)`.

Problem Statement

Creating and Manipulating Arrays: Using NumPy, create a 2D array of shape (3, 4) with random integers between 0 and 10. Perform the following operations: calculate the mean and standard deviation of the entire array, slice the array to get the first two rows and last two columns, reshape the array to shape (4, 3), and perform element-wise multiplication with another array of the same shape.

Working Array: Shape (3, 4)

Create a NumPy array of shape (3, 4) and perform the following five operations:

01

Calculate Mean

Average of all elements

02

Calculate Standard Deviation

Measure of data spread

03

Array Slicing

Extract a subset of elements

04

Array Reshaping

Change dimensions without altering data

05

Element-wise Multiplication

Multiply corresponding elements

Mean of an Array

Definition

Mean is the average of all elements in the array.

Formula

$$\text{Mean} = \frac{\sum X}{N}$$

NumPy Function

```
np.mean(arr)
```

Working Example

```
[[ 2  5  8 10]
 [ 7  1  3  9]
 [ 4  6  2  8]]
```

Calculation

$$\frac{65}{12} = 5.42$$

Output

```
5.42
```

Sum of all 12 elements = 65, divided by 12 elements.

Standard Deviation

What is Standard Deviation?

Measures the spread of data around the mean. A low value means data is clustered near the mean; a high value means data is widely spread.

NumPy Function

```
np.std(arr)
```

Example

```
std_val = np.std(arr)
print(std_val)
```

Output

```
2.96
```

This indicates moderate variation in the data — values are neither tightly clustered nor extremely spread out.

Interpretation Guide

- Low value → Data close to mean
- High value → Data widely spread

Array Indexing & Slicing

Indexing — Access a Single Element

```
arr[0, 0]   # Output: 2
```

General form: `arr[row, column]`

Slicing Syntax

```
arr[row_range, column_range]
```

Example

```
arr[:2, -2:]
```

- **:2 → First 2 rows**
- **-2: → Last 2 columns**

Original Array

```
[[ 2  5  8 10]
 [ 7  1  3  9]
 [ 4  6  2  8]]
```

Sliced Output

```
[[ 8 10]
 [ 3  9]]
```

Only the intersection of the first 2 rows and last 2 columns is returned.

Reshaping Arrays

Reshaping changes the dimensions of an array without altering its data or total number of elements.

1. **Original Shape**
   `(3, 4)` — 3 rows, 4 columns
2. **Apply Reshape**
   `arr.reshape(4, 3)`
3. **New Shape**
   `(4, 3)` — 4 rows, 3 columns

Reshape: Output & Observation

Original Array — Shape (3, 4)

```
[[ 2  5  8 10]
 [ 7  1  3  9]
 [ 4  6  2  8]]
```

Code

```
reshaped_arr =
arr.reshape(4, 3)
print(reshaped_arr)
```

Reshaped Array — Shape (4, 3)

```
[[ 2  5  8]
 [10  7  1]
 [ 3  9  4]
 [ 6  2  8]]
```

Key Observation

All 12 elements remain unchanged. Only the arrangement into rows and columns is modified. Total elements must match: 3×4 = 4×3 = 12.

Element-wise Multiplication

Multiply corresponding elements of two arrays of the same shape. This is different from matrix multiplication.

Array A

```
[[2  5]
 [3  4]]
```

Array B

```
[[1  2]
 [5  6]]
```

Result (A × B)

```
[[ 2  10]
 [15  24]]
```

Code

```
arr2 = np.random.randint(
  1, 6, size=(4, 3)
)
result = reshaped_arr * arr2
```

The `*` operator performs element-wise multiplication in NumPy.

Multiplication: Sample Output

Array 1 — Reshaped (4×3)

```
[[ 2  5  8]
 [10  7  1]
 [ 3  9  4]
 [ 6  2  8]]
```

Array 2 — Random (4×3)

```
[[2  1  3]
 [4  2  1]
 [3  5  2]
 [1  4  2]]
```

Result — Element-wise Product

```
[[ 4   5  24]
 [40  14   1]
 [ 9  45   8]
 [ 6   8  16]]
```

Each element in Array 1 is multiplied by the element at the same position in Array 2. Both arrays must have identical shapes.

Complete Program

```
import numpy as np

arr = np.random.randint(0, 11, size=(3, 4))
print("Original Array"); print(arr)

print("Mean"); print(np.mean(arr))
print("Standard Deviation"); print(np.std(arr))
print("Sliced Array"); print(arr[:2, -2:])

reshaped_arr = arr.reshape(4, 3)
print("Reshaped Array"); print(reshaped_arr)

arr2 = np.random.randint(1, 6, size=(4, 3))
print("Second Array"); print(arr2)

result = reshaped_arr * arr2
print("Multiplication"); print(result)
```

Real-World Applications of NumPy

Data Science

Data processing, cleaning, and statistical analysis at scale.

Machine Learning & AI

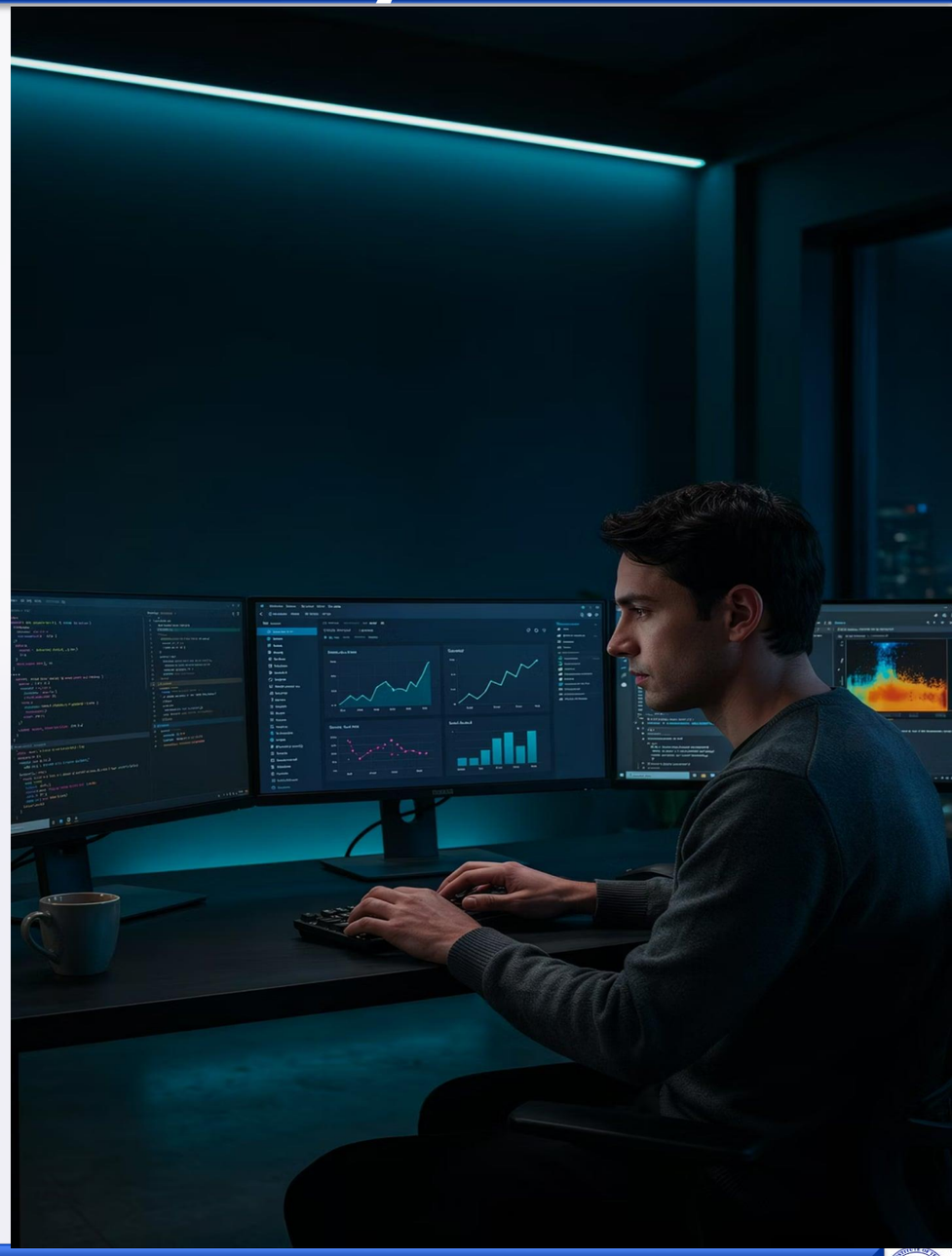
Feature matrices, model training, and neural network computations.

Image Processing

Pixel manipulation — images are stored as NumPy arrays of values.

Scientific Computing

Matrix calculations used in physics, engineering, and simulations.



Summary

Topics Covered

- → Introduction to NumPy
- → Array Creation (1D, 2D, 3D)
- → Random Array Generation
- → Mean & Standard Deviation
- → Indexing, Slicing & Reshaping
- → Element-wise Multiplication

Key Benefits of NumPy

- Faster computation than Python lists
- Efficient memory usage
- Powerful built-in math functions
- Easy multi-dimensional manipulation
- Foundation for Pandas, Scikit-Learn, TensorFlow

Try It Yourself

Create a NumPy array of shape (4, 5) and perform the following operations:

1

Calculate Mean

Use `np.mean()`

2

Find Maximum Value

Use `np.max()`

3

Extract Last Two Rows

Use slicing: `arr[-2:, :]`

4

Reshape to (5, 4)

Use `arr.reshape(5, 4)`

5

Element-wise Addition

Add with another `(5, 4)` array using `+`

Summary

- **Introduction to NumPy**
  - Overview of NumPy and its applications
  - Advantages of NumPy over Python lists
  - Numerical and scientific computing concepts
- **NumPy Arrays and Their Structure**
  - Understanding arrays
  - 1D, 2D, and 3D arrays
  - Array creation using np.array()
  - Random array generation using np.random.randint()
- **Statistical Operations on Arrays**
  - Calculating mean using np.mean()
  - Calculating standard deviation using np.std()
  - Interpreting statistical measures
- **Array Manipulation Techniques**
  - Array indexing and slicing
  - Reshaping arrays using reshape()
  - Understanding dimensions and shape
- **Element-wise Array Operations**
  - Element-wise multiplication
  - Working with arrays of identical shapes
  - Comparing element-wise operations with matrix operations
- **Applications of NumPy**
  - Data Science
  - Machine Learning and AI
  - Image Processing
  - Scientific Computing